Microcontroller selection

The microcontroller selection process was a key process in the project, as it would determine the accuracy and functionality of our product. Selecting an appropriate microcontroller was essential, as it would be responsible for reading all sensor measurements, managing power, formatting data packets and communication between the beehive sensing node and the receiving hub **[1]**.

The system will use two Raspberry Pi Pico variants. The beehive sensing node will be a standard Raspberry Pi Pico, and the receiver hub will use a Raspberry Pi Pico W. The Pico at the beehive node will handle local sensor control and readings, data formatting and HC-12 radio transmission. The Pico W will enable access to the dashboard via a local WiFi connection. The Raspberry Pi Pico and W draw approximately 20 mA in their active operation, however, both Pico's support sleep modes, reducing current draw to a maximum of 5 mA. In terms of cost, the Pico is projected to cost $10, while the Pico W will cost nearly $18 **[2]**. As for their functionality, both models contain 26 GPIO pins, I2C buses, 2 UART interfaces, and 3 ADCs, which will provide us with sufficient connectivity for the whole system. [**3]** Additionally, the project will be built around AustSTEM's Kookaberry MicroPython firmware, which runs on the RP2040. The Pico W also contains a CYW43439 Wi-Fi chip, this will allow the receiver hub to host a local web dashboard. (Appendix B)

Sensor Selection

The sensors were selected to provide an accurate and extensive overview of hive health and environmental conditions while accounting for the constraints of the project's cost, power, and size. Specifically, each sensor was evaluated and compared to alternative options with a focus on being compatible with our Pico microcontrollers. Finally, all sensors provided by AustSTEM were cross-referenced with their datasheets to confirm suitability before proceeding to the next phase.

DHT22 - Temperature and humidity sensor

A main feature of our system is to measure both internal temperature and humidity. AustSTEM provided our group with a DHT22. The DHT22 works using a capacitive humidity sensor and a thermistor to measure the surrounding air and send the measurement to the data pin. The DHT22 is an ideal option when it comes to measuring ambient temperature and relative humidity inside and around the hive, especially as these parameters are critical in assessing the colony's ability to maintain a stable 33–36°C **[4]**. (Appendix C)

BME280 – Barometric Pressure, Temperature and Humidity Sensor

To measure the surrounding conditions of the hive, a BME280 will be used. This measures conditions such as barometric pressure, temperature, and humidity, which will alert the user to weather fluctuations which can impact bee behaviour. The BME280 measures atmospheric changes using a piezoresistive element, temperature using an on-die bandgap temperature sensor, and moisture in air using a hygroscopic dielectric material between two electrodes **[5]**. The BME280's I2C interface allows it to share the bus with other sensors without requiring any additional pins. (Appendix D)

VEML770 - ambient light sensor

The productivity of bees can be significantly impacted by sunlight exposure [7]. Light data can be utilised to monitor these beehive conditions as well as solar panel charging conditions. The VEML7700 works by using a photodiode array as well as an integrated analogue-to-digital converter, which communicates readings via an I2C interface. (Appendix E)

INA-219 - Current and Power Monitor

Requirements for low power use make it essential to monitor the current draw and power consumption for the beehive node. By using an INA219, we will be able to measure the current draw and power consumption of the beehive node **[6]**. This data will be displayed to users on the dashboard indicating battery status and will be used to make power management decisions. This includes a low power mode when the battery level drops below a pre-set threshold.

HX711 + Load Cell – Hive Weight Sensor

Measuring the hive's weight can help in determining the status of several hive conditions simultaneously **[7]**. Signs such as a gradual increase in weight can indicate a healthy hive with active honey production, while a sudden drop may signal swarming, theft, or predator disturbance. The HX711 consists of an analogue-to-digital converter that is designed to be used with strain-gauge load cells. When weight is applied, the load cell generates a small and proportional voltage, which the HX711 amplifies and converts to a digital value transmitted to two GPIO pins. (Appendix F)

Power Management

Effective power management is essential for the design of our project given the system must operate continuously for 72 hours without any solar input. To achieve this, the system will use a duty-cycle-based sleep design. The Pico and nearly all the connected components have a sample period of 10 minutes, meaning that the system will alternate between an active sampling state and a low-power sleep every 10 minutes, while the sensors will be powered on for a limited period. This ensures power is only consumed when necessary. (Appendix G) shows the detailed components' power demand and battery estimation.

Duty-Cycle Architecture and Timing

Since the system has a sampling rate of 10 minutes, there will be about 432 complete cycles over our target of 72 operating hours. To save power, not all components will be on for the same amount of time as for instance, the Pico and Kookaberry breakout board will remain on for 75 seconds to complete all sensor readings, format data packets and transmit them via the HC-12 radio to the hub, then enter a low-power sleep mode for the remainder of the 10 minutes. In this phase, the Pico is expected to draw 14.7mA while active and 4.2mA while dormant, averaging 5.51mA and 18.19mW per cycle. Overall this 10-minute cycle approach will effectively represent data with low power consumption, allowing us to achieve our goal of achieving 72 operating hours.

Power contribution

Since all components will be running on 3.3V, supplied by the Pico's onboard regulator, the average power consumption of each component can be found using $P = 3.3(I_{avg})$. (Appendix G) shows the average power consumption of each component.

The GPS module is treated as a special case, this is due to its significantly higher current and power demand. The GPS NEO-6M requires about 67 mA and nearly 60 seconds to connect to a satellite, in which the team came to the conclusion that it would be very impractical to include in our 10-minute cycle. In response, the GPS will have a sampling period of 1440 minutes (once per day) which will lower the average current drawn to only 11mA and thus have a power consumption of 36mW. Conversely, if the GPS were to run on our 10-minute cycle, it would add about 67mA to the system’s total current needed, thus making the 72-hour target nearly impossible with our selected battery.

Receiving Node

The receiving hub will be powered continuously via a standard micro-USB and wall adapter, meaning that it requires no duty-cycle power management strategy. Consequently, the Pico W will remain active at all times to receive incoming radio packets, host the web dashboard over Wi-Fi, and write data to local memory without interruption. By having no sleep modes or active intervals, power consumption is not a design constraint for this part of the system.

Dashboard Design

The second part of the system is the receiver hub, which consists of a web dashboard and will serve as a primary interface through which the users will interact with the system.

Hardware and Architecture

The receiver hub will consist of a casing that contains the Raspberry Pi Pico W as well as an HC-12 radio module, which will be connected directly to its GPIO pins. The receiver hub will be powered using a standard micro-USB cable and a wall adapter. The Pico W will run a MicroPython script that will perform three tasks. These included receiving and parsing incoming radio packets from the beehive node, storing the data in local memory, and hosting a dashboard over Wi-Fi. The Pico will be configured to expect a single string containing multiple data readings arranged in a specific sequence. The table below maps out the structural format of the transmitted telemetry packet. (Appendix H)

Header ID	Temp /Hum	Light (lux)	Mass (kg)	Audio Peak freq(Hz)	GPS (Latitude)	GPS (longitudinal)	End code
#Hive	24 / 62	25000	20	300Hz	-20, 130	130	\r\n

Table 5

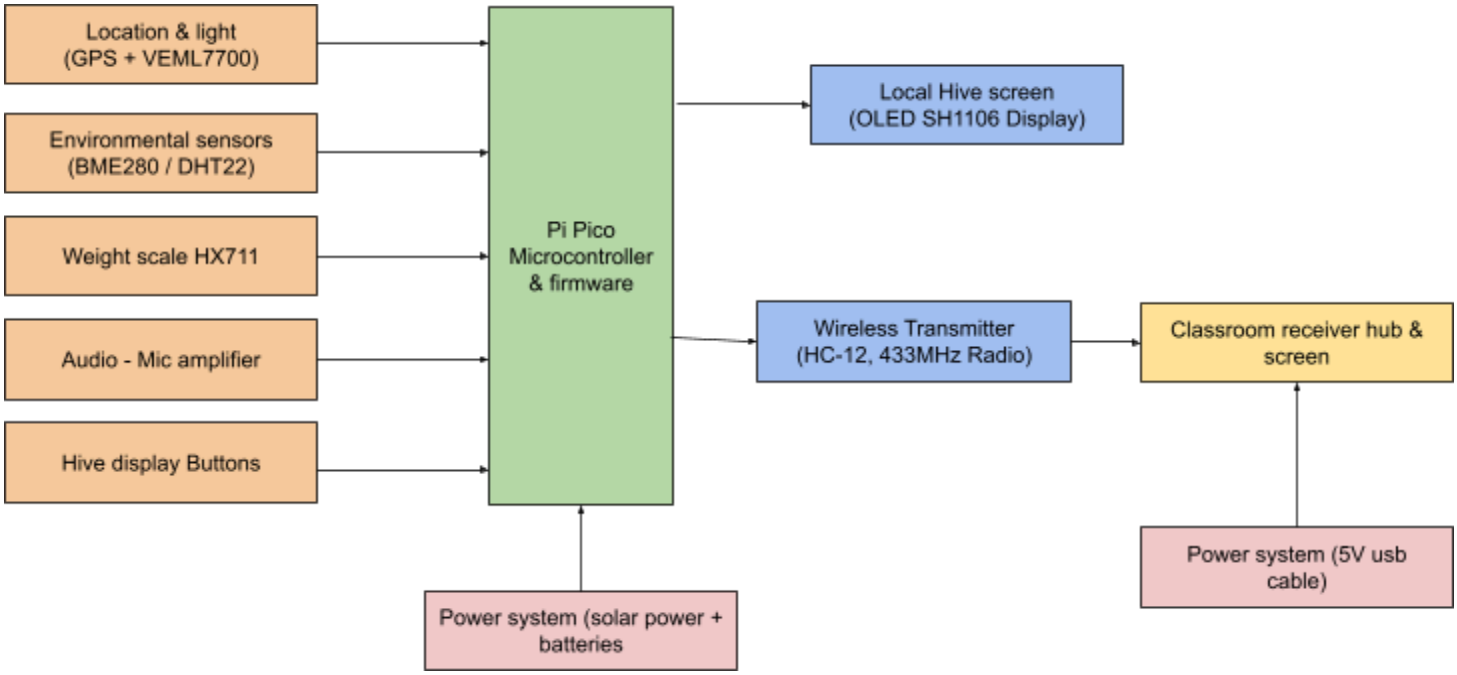
Any device on the same network will be able to access the sensor node data, as users will be able to request the current data dashboard through a browser with no internet required. This keeps the whole system self-contained and easy to use in any location.

User Interface

The dashboard will have a very simple landing page layout, displaying the output of each sensor in a colour-coded tile format. These values will be updated to the latest readings when the UI is accessed. Right under the live cards, each sensor will have a

graph showing historical data with adjustable time scale, allowing users to visualise how conditions inside and outside the beehive can trend over time. Status will be automatically identified through data comparisons against user-specified thresholds, such as ‘safe’, ‘danger’, ‘warning’ statuses. For example, if the temperature of the hive exceeds the specified temperature range, the temperature card will shift from green to red as a clear visual warning to check the beehive or sensors. The dashboard will include a "Hive Health" score feature, a custom algorithm that will aggregate an overall health rating for the beehive on a scale from 0 to 100. Finally, the dashboard will also recommend improvements to improve the health of the hive based on sensor readings. giving users a real-time overview of the overall hive health. (Appendix I) shows the expected layout of our dashboard.

General System overview



Classroom hub:

The classroom hub will be configured to expect a single string incoming with multiple data components arranged in a strict and predictable sequence. The figure below maps out the structural blueprint of the transmitted telemetry packet.

Header ID	Temp /Hum	Light (lux)	Mass (kg)	Audio Peak	GPS (Latitude)	GPS (longitudinal)	End code
#Hive	24 / 62	25000	20	60dB	-20, 130	130	\r\n

Parameter	Raspberry Pi Pico	Raspberry Pi Pico W
Microcontroller	RP2040 (dual-core ARM Cortex-M0+)	
Clock Speed	Up to 133 MHz	
Flash Memory	2 MB	
RAM	264 KB SRAM	
GPIO Pins	26 multi-function GPIO	
Communication Interfaces	2x UART, 2x SPI, 2x I2C, 16x PWM	
ADC	3x 12-bit ADC inputs	
Operating Voltage	1.8V – 5.5V input, 3.3V logic	
Supply Current (active)	~25 mA typical	
Wi-Fi	NA	2.4 GHz 802.11n via CYW43439
Bluetooth	NA	Bluetooth 5.2 (CYW43439)
Programming Language	MicroPython (Kookaberry firmware)	
Cost (approx.)	~$7–10 AUD	~15–18 AUD

Appendix B: Raspberry Pi Pico and Pico W specifications

Parameter	Value
Supply Voltage	2.6V - 5.5V
ADC resolution	24-bit
Output Data Rate	10 Hz or 80 Hz selectable
Load Cell Capacity	20 kg (typical for hive monitoring)
Interface	Two-wire digital (DATA + CLK GPIO)
Current Draw	~1.5 mA active, ~1 µA power down
Cost (approx.)	~$5–10 AUD

Appendix C: DHT22 specifications

Parameter	Value
Pressure Range	300–1100 hPa
Pressure Accuracy	±1 hPa
Temperature Range	-40°C to +85°C
Humidity Range	0–100% RH
Supply Voltage	1.71V – 3.6V

Interface	I2C (address 0x76 or 0x77) / SPI
Current Draw	~3.6 µA at 1 Hz sampling
Cost (approx.)	~$8 - 12 AUD

Appendix D: BME280 specifications

Parameter	Value
Measurement Range	0 – 120,000 lux
Resolution	16-bit
Supply Voltage	2.5V – 3.6V
Interface	I2C (address 0x10)
Current Draw	~8 µA active, ~0.5 µA shutdown
Spectral Response	Visible light (peaked ~550 nm)
Cost (approx.)	~$6–10 AUD

Appendix E: VEML770 specifications

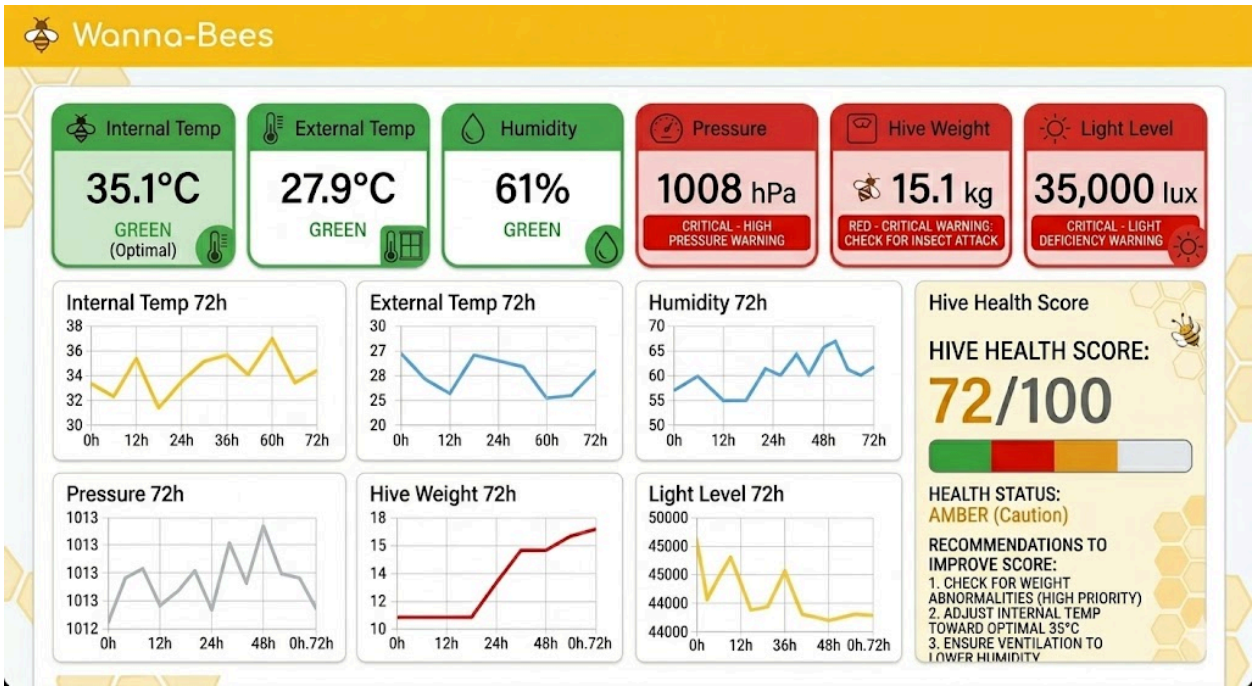
Parameter	Value
Supply Voltage	3.0V – 5.5V
Shunt Voltage Range	±40 mV (programmable)
Bus Voltage Range	0 – 26V
Current Resolution	0.1 mA (with 0.1Ω shunt)
Interface	I2C (address 0x40–0x4F, configurable)
Current Draw	~13mA active, ~6 µA standby
Cost (approx.)	~$5–8 AUD

Appendix F: HX711 + load cell specifications

	A	B	C	D	E	F	G	H	I	J	K	L	M
1	**Component**	**Timing Base**	**Sample Period**	**Seconds on**	**Seconds run in 72 hours**	**seconds dormant**	**Current**				**Battery**		**Power - P = VI (V=3.3)**
2			10 means it turns on once every 10 mins	seconds per period	259200 seconds = 432 cycles	T-tactive	active	dorment	average	total	capacity in mAh	Ah	
3	Pico + Quokka breakout	cycle window	10	75	32400	226800	14.7	4.2	5.5125	36.68659321	2641.434711	2.641434711	18.19125
4	DHT22 ×2	sample	10	4	1728	257472	1.5	0.05	0.05966666667				0.1969
5	BME280	sample	10	1	432	258768	3.60E-06	1.00E-07	1.06E-07		**Batery req**		3.49E-07
6	VEML7700 light	sample	10	1	432	258768	8.00E-06	5.00E-07	5.13E-07		usable factor		1.69E-06
7	ENS160/AHT21 CO2/temp/humidity	sample/warm up	10	60	25920	233280	29		2.9		0.8	5.826694216	9.57
8	HX711 + load cell	sample	10	5	2160	257040	13	0	0.1083333333		efficiency		0.3575
9	Audio/mic amplifier	sample burst	10	10	4320	254880	10		0.1666666667		0.85		0.55
10	INA219 power monitor	sample	10	1	432	258768	1		0.001666666667		safty factor		0.0055
11	Micro SD reader	logging	10	2	864	258336	0	0	0		1.5		0
12	HC-12 radio	transmit	10	6	2592	256608	100	16	16.84				55.572
13	GPS NEO-6M	periodic	1440	60	180	259020	67	11	11.03888889		**Bat actual**		36.42833333
14	OLED SH1106	User triggered	10 uses per day	20	(10*72/24)*20 = 600	258600	25		0.05787037037		6.4		0.1909722222
15	Button board / PB-LED	status blink	10	0.2	86.4	259113.6	3		0.001		SF=	1.647589464	0.0033
16	TP4056 solar charger	N/A	0	0	0	0	0		0				

Appendix G: Components' current and power consumption

Appendix H: Receiving node design



Appendix I: AI-generated dashboard of expected dashboard layout